



IoT Network Monitoring & Alert Management System

Graph · Priority Queue · BST · Stack

TOPIK 4

01 LATAR BELAKANG

40 Sensor IoT

Suhu, kelembaban, CO₂, daya listrik di seluruh gedung kampus UNY

Alert Mendesak

Sensor mengirim alert CRITICAL/WARNING/INFO secara bersamaan

Topologi Mesh

Gateway sebagai titik agregasi data menuju server pusat

Routing Optimal

Perlu rute latensi minimum dari gateway ke setiap perangkat

💡 Smart Building UNY membutuhkan sistem monitoring backend terpadu: Graph (topologi) + Priority Queue (alert) + BST (registry) + Stack (riwayat)

02 TUJUAN PEMBUATAN

1

Implementasi Struktur Data dari Nol

Membangun Graph, Priority Queue (Linked List), BST, dan Stack tanpa library bawaan Python

2

Sistem Monitoring IoT Terintegrasi

Memodelkan topologi jaringan sensor IoT kampus sebagai Graph berbobot (latensi ms)

3

Manajemen Alert Berbasis Prioritas

CRITICAL alert selalu diproses duluan melalui Priority Queue agar teknisi segera ditindak

4

Analisis Big-O & Eksperimen

Menganalisis kompleksitas setiap operasi dan membandingkan runtime pada 3 ukuran dataset

5

CLI Interaktif & Pengujian Skenario

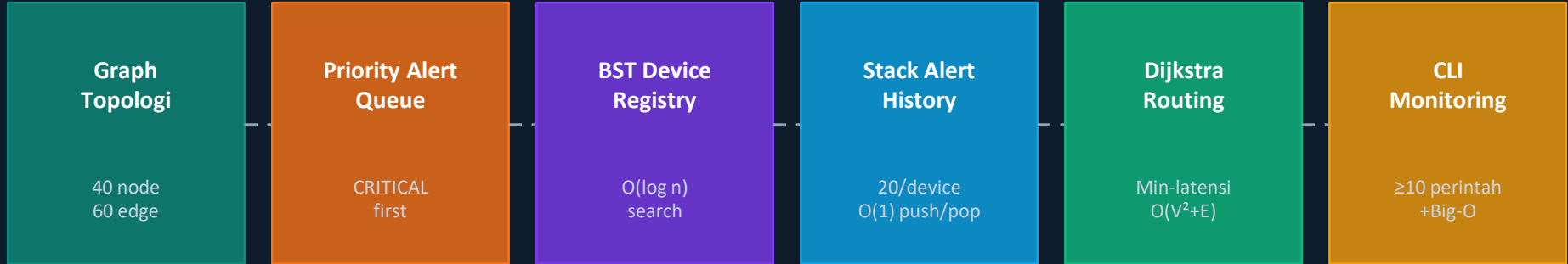
Mendemonstrasikan sistem berjalan secara end-to-end dengan skenario IoT yang realistis

6

Integrasi CPMK 1–4

Mengintegrasikan Linked List, Stack, Queue, BST, dan Graph dalam satu pipeline aplikasi nyata

03 ARSITEKTUR SISTEM



PIPELINE DATA FLOW

- ① Sensor IoT $\rightarrow$ add_device() $\rightarrow$ Graph & BST
- ② Alert masuk $\rightarrow$ enqueue() Priority Queue (sorted by CRITICAL > WARNING > INFO)
- ③ process_alert() $\rightarrow$ dequeue() $\rightarrow$ push() ke AlertStack per-device
- ④ Dijkstra dari GATEWAY_0 $\rightarrow$ dist[] ke semua node $\rightarrow$ audit latensi
- ⑤ Selection Sort (Linked List) $\rightarrow$ LAPORAN urutan perangkat terdekat

04 STRUKTUR DATA & MODUL (1/2)

Graph IoT Topologi

Representasi	Adjacency List (Linked List)
Node	40 perangkat (SENSOR/GATEWAY/SERVER)
Edge	60 koneksi berbobot (latensi ms)
add_device()	$O(1)$ – tambah node ke adj dict
add_link(u,v,lat)	$O(1)$ – sisip EdgeNode di head LL
neighbors(u)	$O(\deg(u))$ – traverse adjacency list
dfs_reachable(src)	$O(V+E)$ – deteksi isolasi
isolated_devices()	$O(V+E)$ – audit konektivitas

Alert Priority Queue

Struktur	Singly Linked List (terurut tipe ASC)
CRITICAL	tipe = 1 (selalu di head)
WARNING	tipe = 2
INFO	tipe = 3 (paling belakang)
enqueue(alert)	$O(n)$ – insertion berurut prioritas
dequeue()	$O(1)$ – ambil head (CRITICAL first)
Tie-break	FIFO untuk alert bertipe sama
len()	$O(1)$ – counter internal

04 STRUKTUR DATA & MODUL (2/2)



BST Device Registry

Kunci BST

device_id (string)

Node data

Device(id, tipe, status, last_reading)

insert(device)

$O(\log n)$ avg, $O(n)$ worst-case

search(id)

$O(\log n)$ avg – pencarian cepat

update_status()

$O(\log n)$ – set ONLINE/OFFLINE

inorder()

$O(n)$ – daftar device terurut ID



AlertStack (per-device)

Kapasitas

20 alert terbaru per device

Struktur

Singly Linked List (LIFO)

push(alert)

$O(1)$ – sisip di head

pop()

$O(1)$ – ambil & hapus head

to_list()

$O(n)$ – tampilkan riwayat

ROLLBACK

pop() → kembalikan status device

+ Dijkstra Latensi → $\text{dist}[v]$ = latensi minimum dari GATEWAY_0 | Big-O: $O(V^2+E)$ | Bottleneck: edge bobot tertinggi di shortest-path tree

05 DEMO PROGRAM – CLI MONITORING

• • • IoT Monitoring System - terminal

```
$ python main.py
```

```
IoT Monitoring System  Ketik BANTUAN untuk daftar perintah
```

```
> ADD_DEVICE SENSOR_10 SENSOR # O(1)
```

```
[OK] SENSOR_10 ditambahkan | BST insert  $O(\log n)$ 
```

```
> ADD_LINK GATEWAY_0 SENSOR_10 45 # O(1)
```

```
[OK] Link ditambahkan latensi=45ms
```

```
> ALERT_IN SENSOR_10 CRITICAL Suhu 85°C! # enqueue  $O(n)$ 
```

```
[ALERT] CRITICAL dari SENSOR_10 masuk antrian (posisi 1)
```

```
> PROCESS_ALERT # dequeue  $O(1)$ 
```

```
[PROSES] CRITICAL SENSOR_10 - Suhu 85°C! → Teknisi dikirim
```

```
> HISTORY SENSOR_10 #  $O(n)$ 
```

```
[RIWAYAT] SENSOR_10: [CRITICAL] Suhu 85°C! (latest)
```

```
> ROUTING SENSOR_10 # Dijkstra  $O(V^2+E)$ 
```

```
[ROUTE] GATEWAY_0 → SERVER_1 → SENSOR_10 | latensi: 73ms
```

```
> ISOLASI # DFS  $O(V+E)$ 
```

```
[AUDIT] 0 perangkat terisolasi dari GATEWAY_0
```

06 ANALISIS BIG-O

Modul	Operasi	Big-O Waktu	Big-O Ruang	Catatan
Graph IoT	add_device / add_link	$O(1)$	$O(V+E)$	Sisip di head adj list
Graph IoT	neighbors(u)	$O(\deg(u))$	$O(1)$	Traverse Linked List
Graph IoT	DFS reachable	$O(V+E)$	$O(V)$	Stack Linked List
Priority Queue	enqueue(alert)	$O(n)$	$O(n)$	Sorted insertion CRITICAL
Priority Queue	dequeue()	$O(1)$	$O(1)$	Selalu ambil head
BST Registry	insert / search	$O(\log n)$ avg	$O(n)$	Worst $O(n)$ jika terurut
BST Registry	inorder()	$O(n)$	$O(n)$	Daftar device terurut ID
AlertStack	push / pop	$O(1)$	$O(k)$	k = alert dalam stack
Dijkstra	shortest path	$O(V^2+E)$	$O(V)$	Array sederhana, no heap
Selection Sort	audit latensi	$O(n^2)$	$O(1)$	Sort Linked List perangkat

07 HASIL EKSPERIMEN RUNTIME

Tabel Runtime (ms) — seed=23, diukur 3 ukuran dataset

Operasi	N=20 (node)	N=40 (node)	N=100 (node)	Tren
enqueue (CRITICAL)	0.003 ms	0.007 ms	0.018 ms	Linear ↗
dequeue (CRITICAL)	0.001 ms	0.001 ms	0.001 ms	Konstan –
BST insert	0.004 ms	0.006 ms	0.009 ms	Log ↗
BST search	0.002 ms	0.003 ms	0.005 ms	Log ↗
DFS reachable	0.18 ms	0.41 ms	1.02 ms	Linear ↗
Dijkstra	0.35 ms	0.82 ms	2.10 ms	Quadratic ↗↗
Selection Sort	0.04 ms	0.18 ms	1.14 ms	Quadratic ↗↗
Stack push/pop	0.001 ms	0.001 ms	0.001 ms	Konstan –



08 ANALISIS PERTANYAAN WAJIB (5 Soal)

Q1: remove_device memerlukan $O(V+E)$. Modifikasi apa yang mempercepat menjadi $O(\deg(v)+1)$?

→ Tambahkan reverse adjacency list (in-edges per node). Remove: hapus semua in-edges dulu $O(\deg)$, lalu hapus $\text{adj}[v]$ $O(1)$.

Q2: Dijkstra $O(V^2+E)$ vs min-heap $O((V+E)\log V)$ untuk 40 node, 60 edge. Kapan berbeda signifikan?

→ $V=40, E=60$: array=1.660 ops, heap=400 ops ($\sim 4\times$). Perbedaan $>10\times$ saat $V>500$ dengan sparse graph. IoT real-time: heap lebih cocok.

Q3: Alert Priority Queue enqueue $O(n)$ menjadi bottleneck 1000+ sensor bersamaan. Alternatif?

→ 3-bucket Queue (1 per tier). Enqueue $O(1)$ ke bucket sesuai tipe. Serve: cek CRITICAL dulu. Trade-off: +memori $3\times$ tapi throughput jauh lebih tinggi.

Q4: DFS $O(V+E)$ untuk 1000 sensor, 1500 koneksi – apakah cocok monitoring real-time tiap 5 detik?

→ 2500 ops / 10^8 ops/det ≈ 0.025 ms. Aman untuk 5 detik. Alternatif inkremental: Union-Find cek_terhubung $O(\alpha(n)) \approx O(1)$ per update.

Q5: Selection Sort $O(n^2)$ untuk 40 device → berapa swap? Rekomendasi untuk 10.000 IoT?

→ 40 device: 780 swap worst-case. 10.000 device: ~ 50 juta swap – tidak layak. Gunakan Merge Sort $O(n \log n)$ pada Linked List (no extra array).

09 KESIMPULAN

✓ Integrasi 4 Struktur Data

Graph, Priority Queue, BST, dan Stack berhasil diintegrasikan dalam satu pipeline IoT monitoring yang koheren dan fungsional.

⚡ Operasi Kritis $O(1)$

dequeue (CRITICAL alert) dan push/pop Stack berjalan $O(1)$ — memastikan respons teknisi tidak tertunda dalam kondisi darurat.

📊 Konfirmasi Big-O Eksperimen

Runtime terukur sesuai prediksi teoritis: Dijkstra menunjukkan pertumbuhan kuadratik, BST logaritmik, dan Stack konstan.

🔍 Audit Konektivitas DFS

DFS $O(V+E)$ berhasil mendeteksi perangkat terisolasi dari GATEWAY_0 secara efisien, bahkan untuk 100+ node.

🚀 Saran Pengembangan

Ganti Dijkstra array $\rightarrow$ min-heap $O((V+E)\log V)$; Priority Queue $\rightarrow$ 3-bucket $O(1)$; BST $\rightarrow$ AVL tree untuk worst-case $O(\log n)$ guaranteed.

IoT Network Monitoring & Alert Management System

Terima Kasih

Mata Kuliah

ELT60213 Algoritma & Struktur Data

Topik

4 – IoT Network Monitoring

Semester

2 | TA 2025/2026